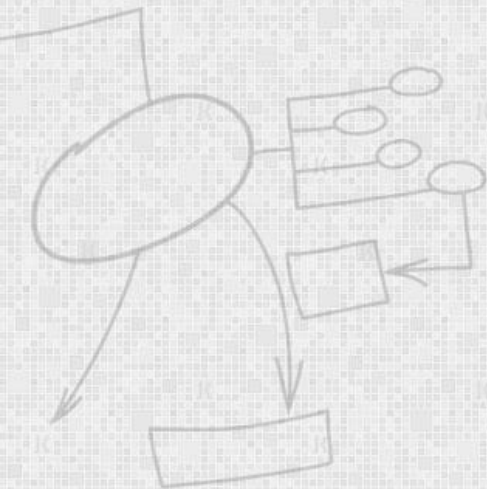


EC-360

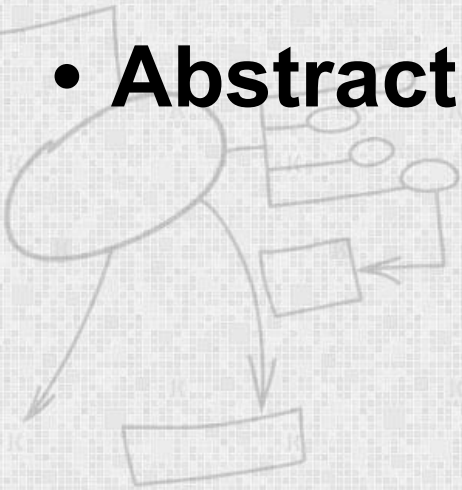
SOFTWARE ENGINEERING

Lecture 1
Introduction
By
Dr Wasi Haider Butt



Today's Lecture: Agenda

- **Introduction of Course**
- **Software Engineering**
- **Need of Software Engineering**
- **Software Development Lifecycle Models**
- **Abstraction and Decomposition**



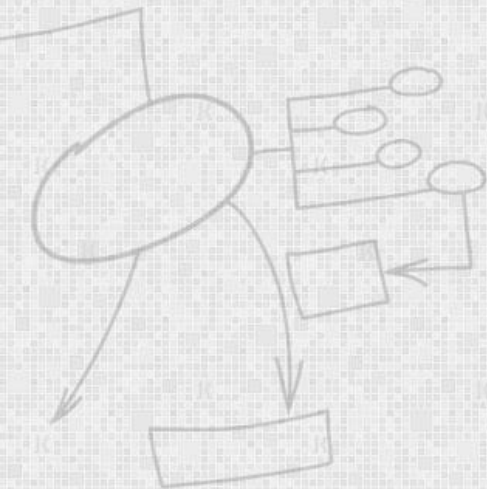
Introduction

- Course Title : Software Engineering
- Code: EC 360
- Instructor : Dr Wasi Haider Butt
- Offered to: Degree 41 CE
- Course Credits: 3-0
- Semester : Fall 2022
- Duration : 18 Weeks (Including Exam Weeks)
- Books: Software Engineering: A practitioner's Approach by Roger S. Pressman, Seventh Edition.
- Reference Material: Will be provided when required



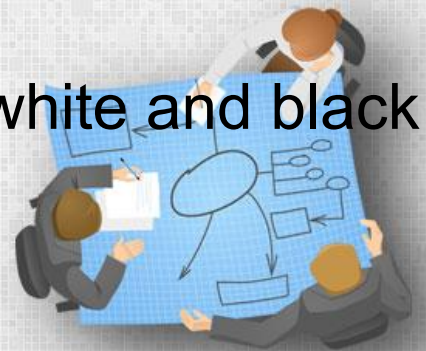
Marks Distribution

• Quizzes	(6)	10%
• Midterm	(1)	30%
• Assignments/Project	(2)	20%
• Final	(1)	40%



Main Objectives

- A general understanding of **software process models** such as the waterfall, evolutionary and agile models like extreme programming and scrum.
- An understanding of **software requirements** and the SRS document.
- An understanding of **object oriented** analysis and design using UML.
- An understanding of implementation issues such as **modularity and coding standards**.
- An understanding of **software testing** including white and black box testing
- An understanding of **software design patterns**



Learning Outcomes

CLO 1	Apply a software development strategy and select appropriate software development model for any given software project	PLO 3	C3
CLO 2	Prepare UML diagrams from requirements to deployment of any given software.	PLO 3	C3
CLO 3	Argue ethical obligations while developing a software for a client organization	PLO 8	A3
CLO 4	Debate on new trends in software development methodologies	PLO 12	A3

Course Contents

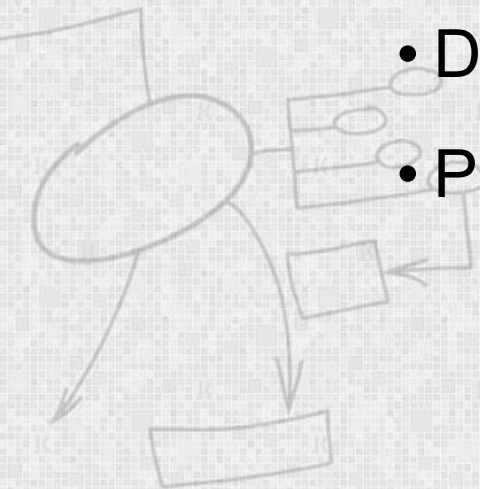
- **Introduction** to Software Engineering
- Software life cycle **models**
- **Agile** software development
- **SRS** Document
- Functional and non-functional **requirements**
- Software **Design**
- Design **Patterns**
- Cohesion/Coupling
- Coding
- Debugging
- Integration
- **White/Black Box** Testing
- Software Maintenance



Software Development Issues

Projects were:

- Late.
- Over budget.
- Unreliable.
- Difficult to maintain.
- Performed poorly.

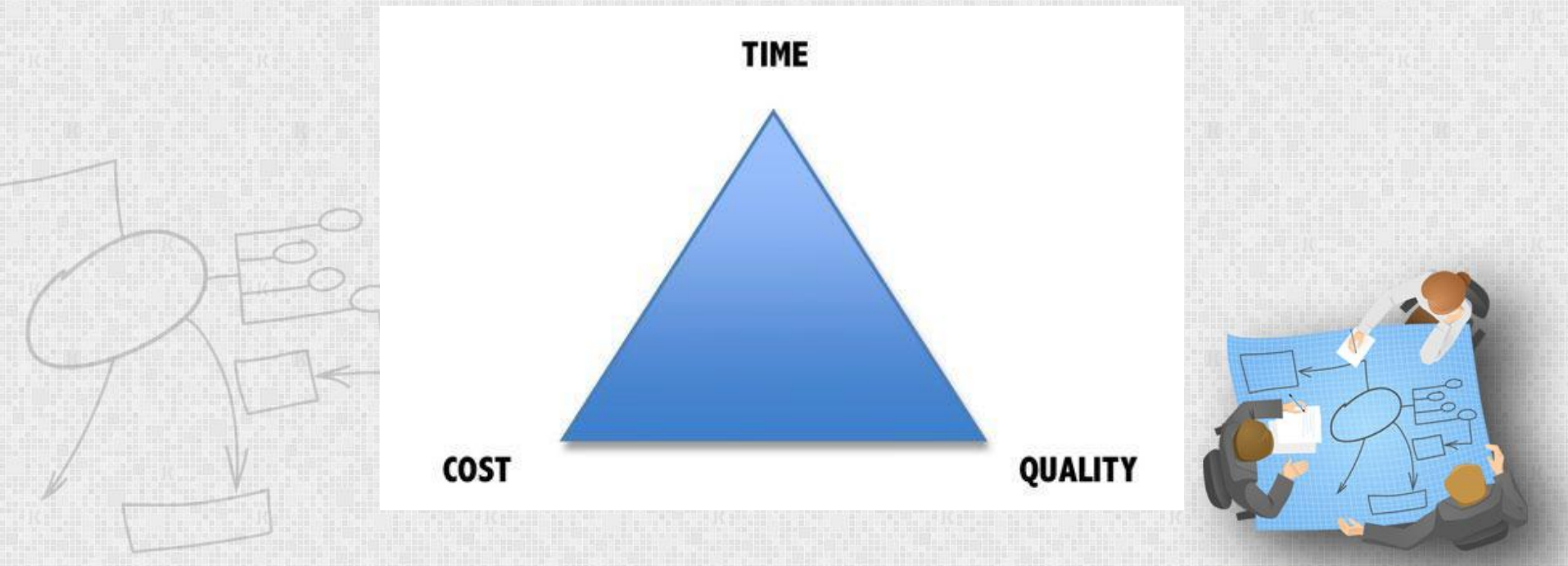


A well-disciplined approach to software development and management is necessary. This is called engineering.



Why is Software Engineering important?

Complex systems need a disciplined approach for designing, developing and managing them.



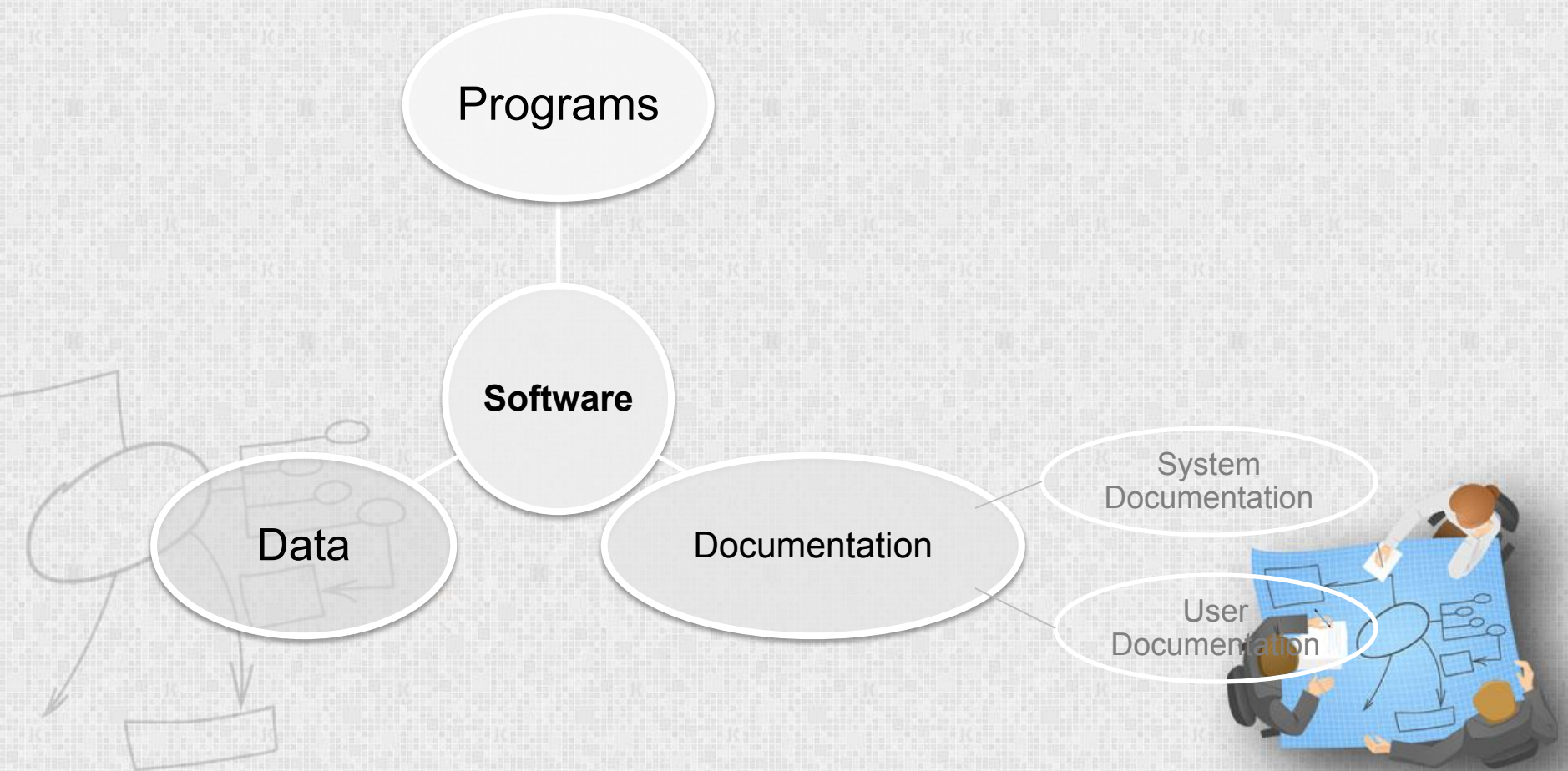
Software Engineering

- ◆ “.. An engineering discipline that is concerned **with all aspects of software production** from the **early stages** of system specification to **maintaining the system** after it has **gone into use.**”

- ◆ Ref: **Ian Sommerville (Software Engineering)**



Software is not only the executable



Professional Responsibility

Organizations put trust in software developers:

- Competence: Software that does not work effectively can destroy an organization.
- Confidentiality: Software developers and systems administrators may have access to highly confidential information (e.g., trade secrets, personal data).
- Legal environment: Software exists in a complex legal environment (e.g., intellectual property).
- Acceptable use and misuse: Computer abuse can paralyze an organization (e.g., the Internet worm).



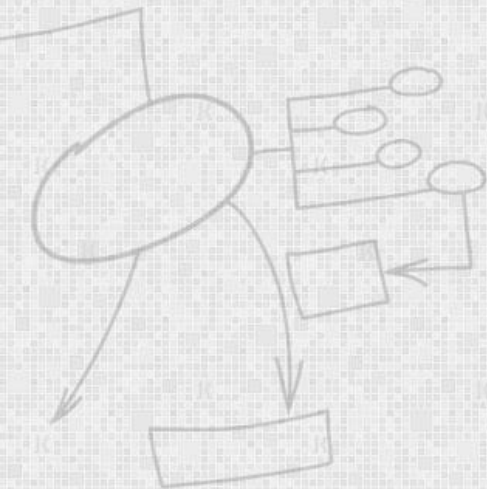
Software Life Cycle Model

- A software life cycle model (also called process model) is a descriptive and diagrammatic representation of the software life cycle.
- A life cycle model represents all the activities required to make a software product.
- It also captures the order in which these activities are to be undertaken.
- In other words, a life cycle model maps the different activities performed on a software product from its start to retirement.



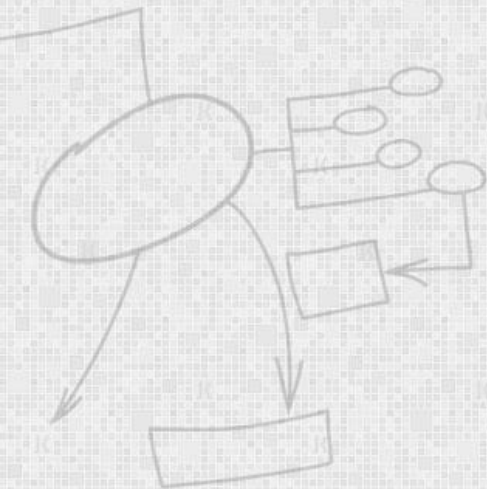
The need for a software life cycle model

- A systematic and disciplined way
- For team management



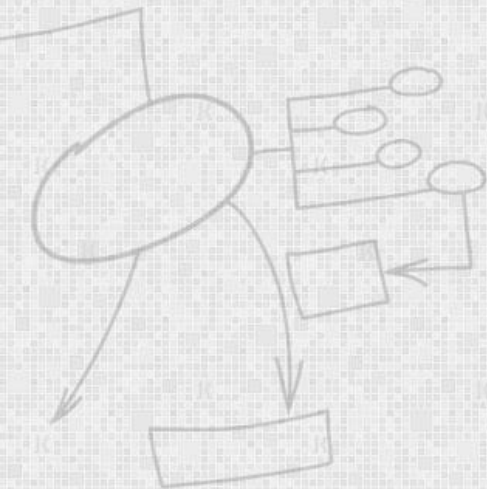
Different software life cycle models

- Few well known
 - Waterfall
 - Prototyping Model or Evolutionary Model



Classical Waterfall Model

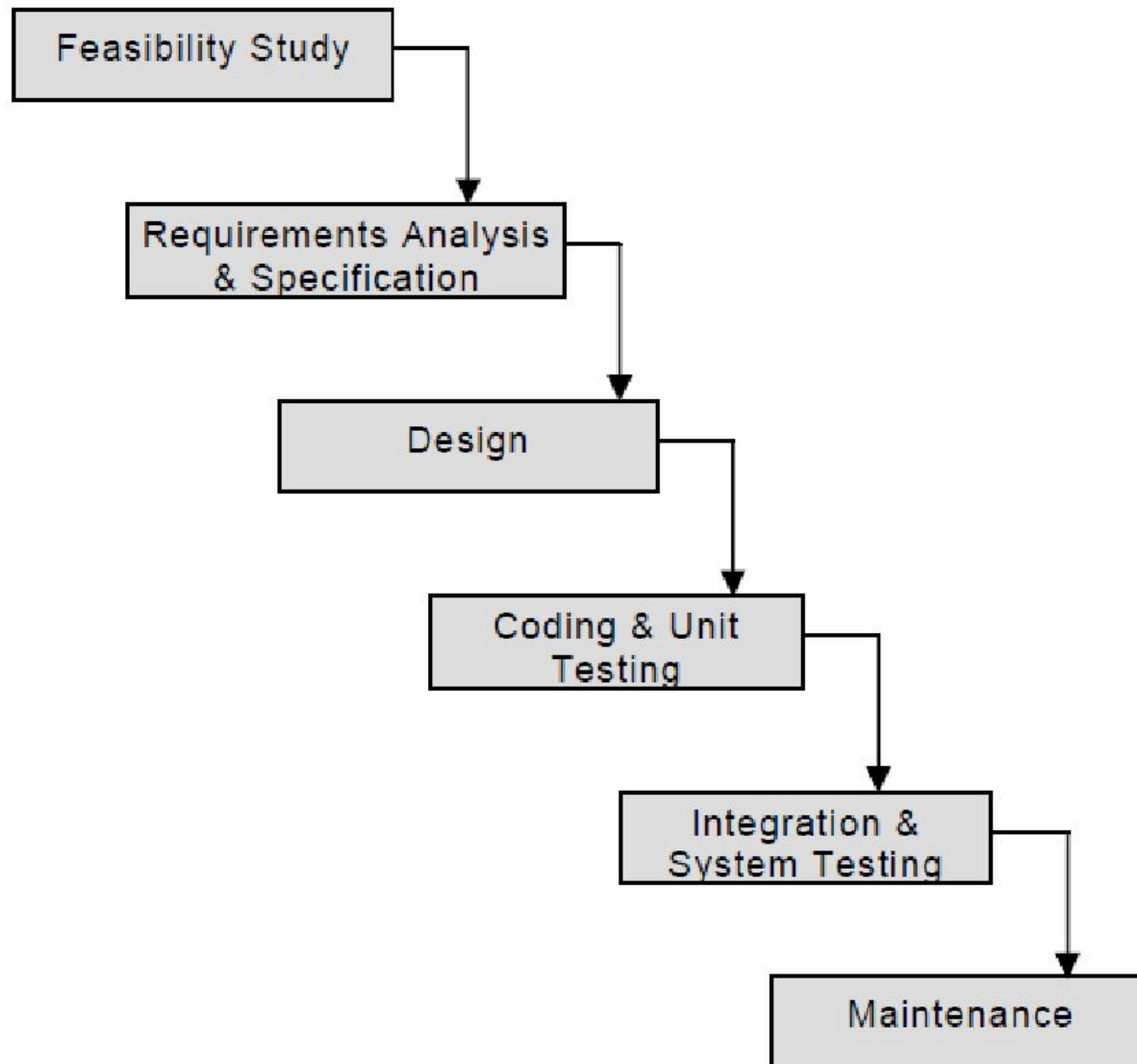
- Most obvious way to develop software.
- *An ideal theoretical way of developing software.*
- All other life cycle models are essentially derived from the classical waterfall model.



Classical Waterfall Model

- Classical waterfall model divides the life cycle into the following phases.
 - Feasibility Study
 - Requirements Analysis and Specification
 - Design
 - Coding and Unit Testing
 - Integration and System Testing
 - Maintenance





Activities undertaken during feasibility study

- Whether it would be financially and technically **feasible** to develop the product.
- To have a rough understanding of what is required to be done by visiting the client side.
- Examining each of the solutions in terms of what kind of resources required, what would be the cost of development and what would be the development time for each solution.
- Based on this analysis pick the best solution and determine whether the solution is feasible financially and technically.



Activities undertaken during requirements analysis & specification

- The aim of the requirements analysis and specification phase is to understand the exact requirements of the customer
- And then to document them properly. This phase consists of
 - Elicitation
 - Analysis
 - Specification
 - Validation



Needs Vs Requirements



NEEDS



REQUIREMENTS

Needs Vs Requirements

Let's take a look at these examples:

Needs (I need to...)	Requirements
build a big house for my large family	• The house shall have 3 bedrooms ◦ 2 small bedrooms ◦ 1 master bedroom • The house shall have a backyard • The house shall be 3 levels • The floors shall be wood ◦ The wood color shall be dark brown
get a BA job	• I should take a BA course • I should create a resume • I should practice mockup interviews
build my portfolio website	• The website shall allow clients to contact me • The website shall have a blog • The website shall present my projects

Activities undertaken during design

- Transform the requirements specified in the SRS (Software Requirement Specification) document into a structure that is suitable for implementation in some programming language.
- Software architecture is derived from the SRS document.



Activities undertaken during coding and unit testing:

- Translating the software design into source code.
- Each component of the design is implemented as a program module.
- The end-product of this phase is a set of program modules that have been individually tested.
- During this phase, each module is unit tested to determine the correct working of all the individual modules.
- It involves testing each module in isolation as this is the most efficient way to debug the errors identified at this stage.



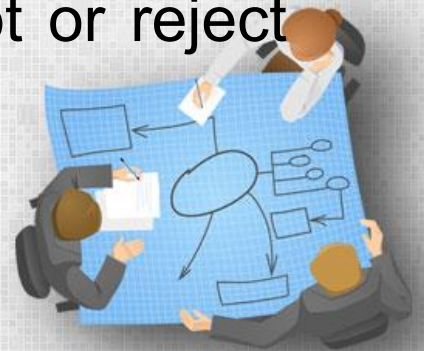
Activities undertaken during integration and system testing:

- The different modules making up a software product are almost never integrated in one shot.
- Integration is normally carried out incrementally over a number of steps.
- During each integration step, the partially integrated system is tested and a set of previously planned modules are added to it.
- Finally, when all the modules have been successfully integrated and tested, system testing is carried out.



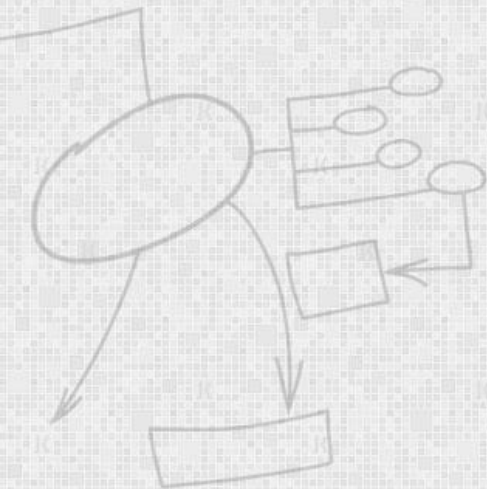
Activities undertaken during integration and system testing:

- System testing usually consists of three different kinds of testing activities:
 - α – testing: It is the system testing performed by the development team.
 - β – testing: It is the system testing performed by a friendly set of customers.
 - acceptance testing: It is the system testing performed by the client himself after the product delivery to determine whether to accept or reject the delivered product.



Activities undertaken during maintenance

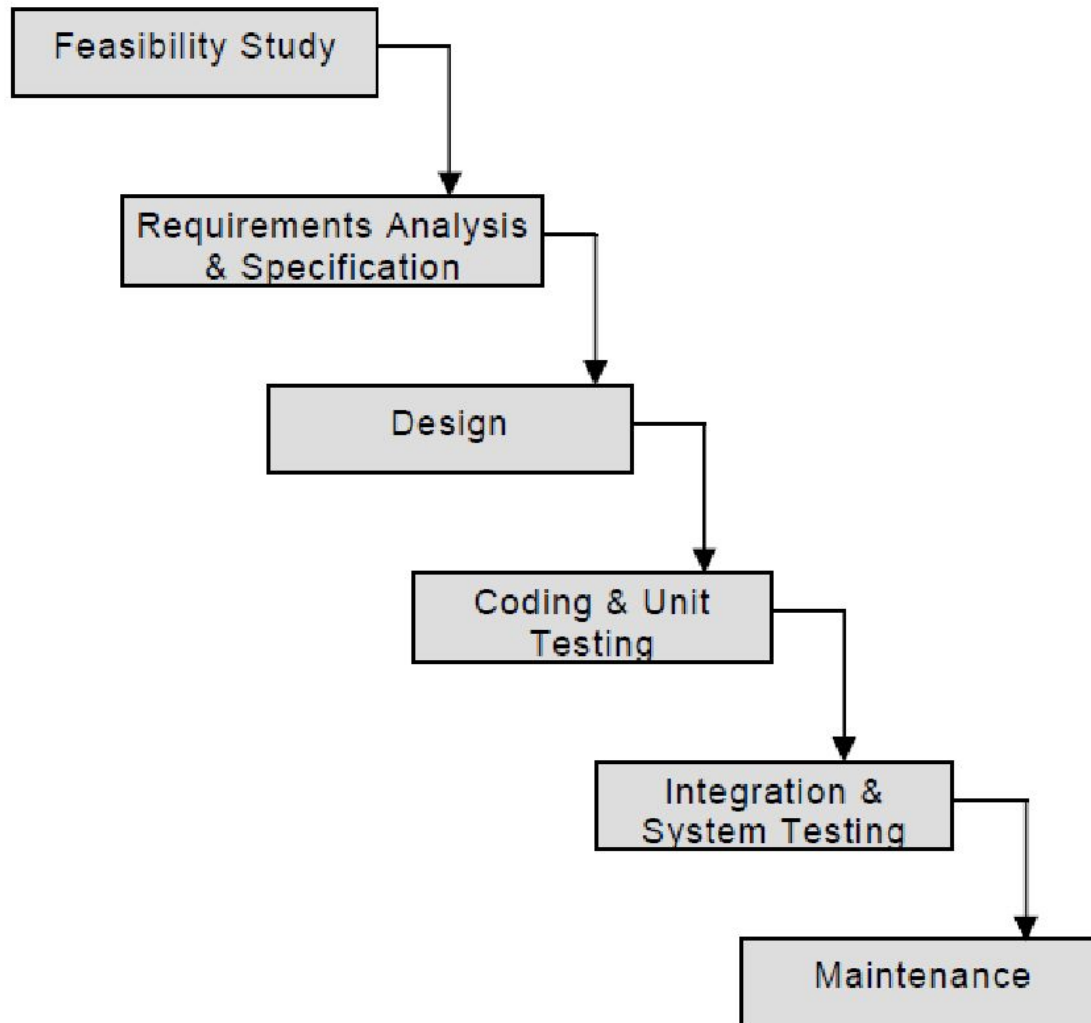
- Maintenance of a typical software product requires much more than the effort necessary to develop the product itself.
- Maintenance effort is roughly in the 40:60 ratio.



Activities undertaken during maintenance

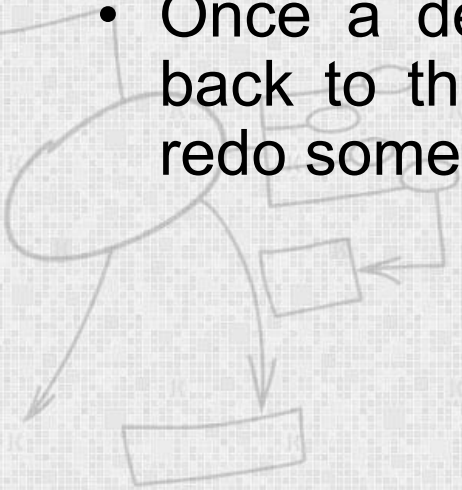
- Maintenance involves performing following kinds of activities:
 - Correcting errors that were not discovered during the product development phase. This is called **corrective maintenance**.
 - Improving the implementation of the system, and enhancing the functionalities of the system according to the customer's requirements. This is called **perfective maintenance**.
 - Porting the software to work in a new environment. For example, porting may be required to get the software to work on a new computer platform or with a new operating system. This is called **adaptive maintenance**.





Shortcomings of the classical waterfall model

- The classical waterfall model is an **idealistic** one since it assumes that **no development error is ever committed** by the engineers during any of the life cycle phases.
- However, in practical development environments, the engineers do commit a large number of errors in almost every phase of the life cycle.
- Once a defect is detected, the engineers need to go back to the phase where the defect had occurred and redo some of the work.



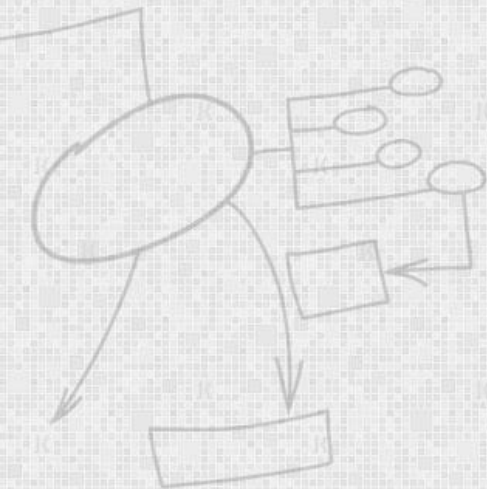
Prototype

- A prototype is a **toy implementation** of the system.
- A prototype usually exhibits limited functional capabilities, low reliability, and inefficient performance compared to the actual software.
- A prototype is usually built using several shortcuts. The shortcuts might involve using inefficient, inaccurate, or dummy functions.
- A prototype usually turns out to be a very crude version of the actual system.
- Throwaway Vs Evolutionary Prototype



Changing nature of software

- Broad categories of software that are evolving to dominate the industry
 - Web Apps
 - Mobile Apps
 - Cloud Computing



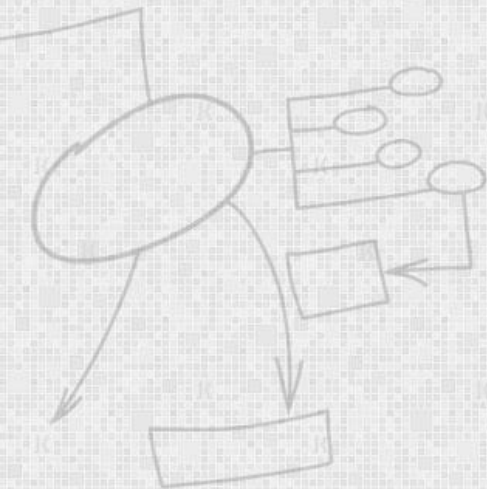
Web Apps

- In the early days of the World Wide Web (circa 1990 to 1995), *websites* consisted of little more than a set of linked hypertext files that presented information using text and limited graphics.
- As time passed, the augmentation of HTML by development tools (e.g., XML, Java) enabled Web engineers to provide computing capability along with informational content.
- *Web-based systems and applications* (we refer to these collectively as *WebApps*) were born.



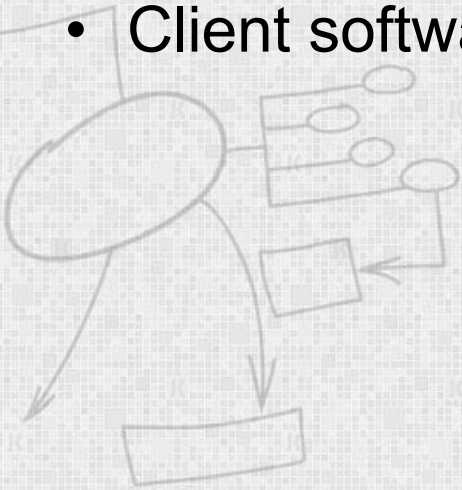
Mobile Apps

- The term *app* has evolved to represent software that has been specifically designed to reside on a mobile platform (e.g., iOS, Android, or Windows Mobile).
- Access to web resources, customized to be best viewed at mobile screen along with local execution enabling use of local resources and storage
- Mobile Web Apps VS Mobile Apps

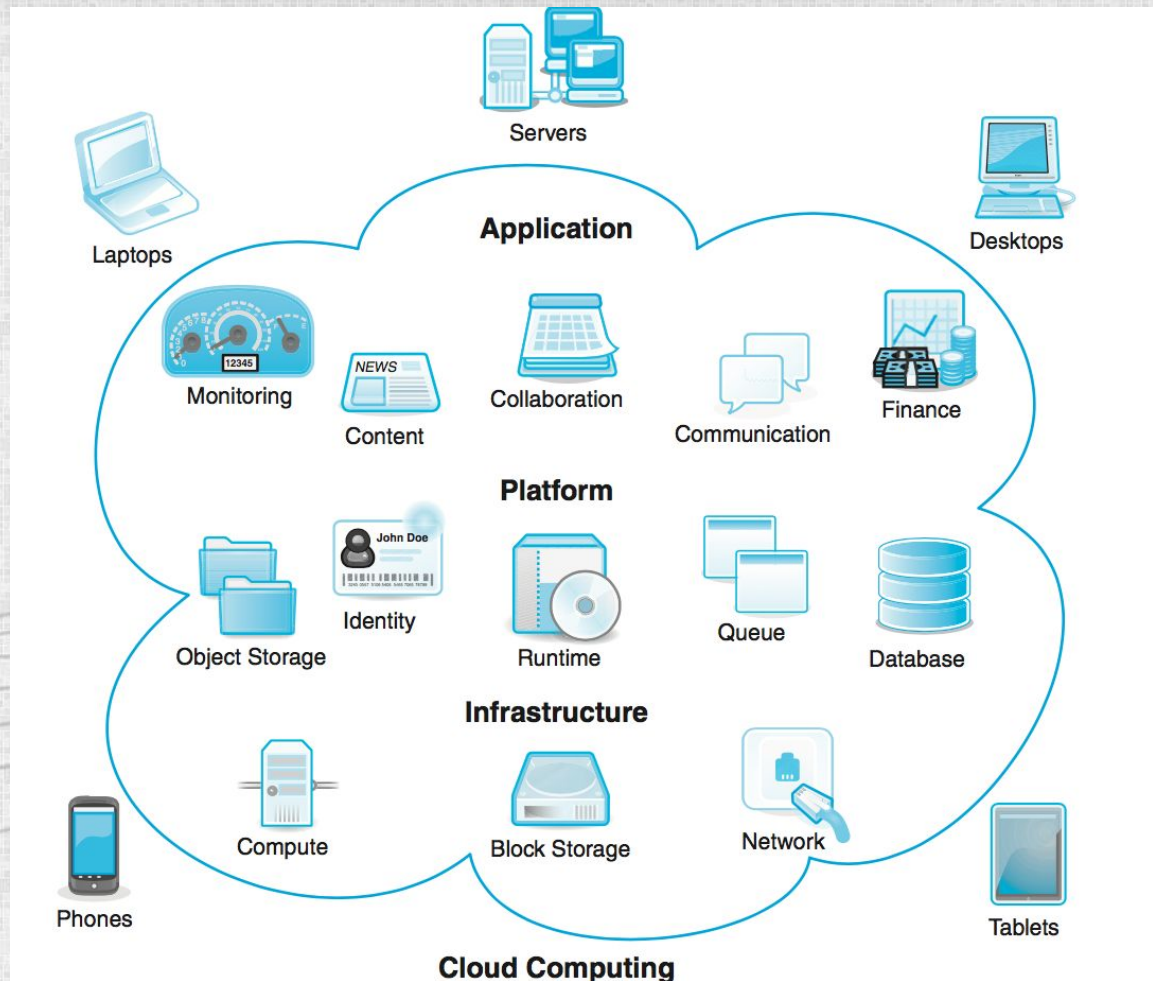


Cloud Computing

- *Cloud computing* encompasses an infrastructure or “ecosystem” that enables any user, anywhere, to use a computing device to share computing resources on a broad scale.
- Users connect to use computing resources
- Simplest form is through web browsers
- Client software can also be used



Cloud Computing

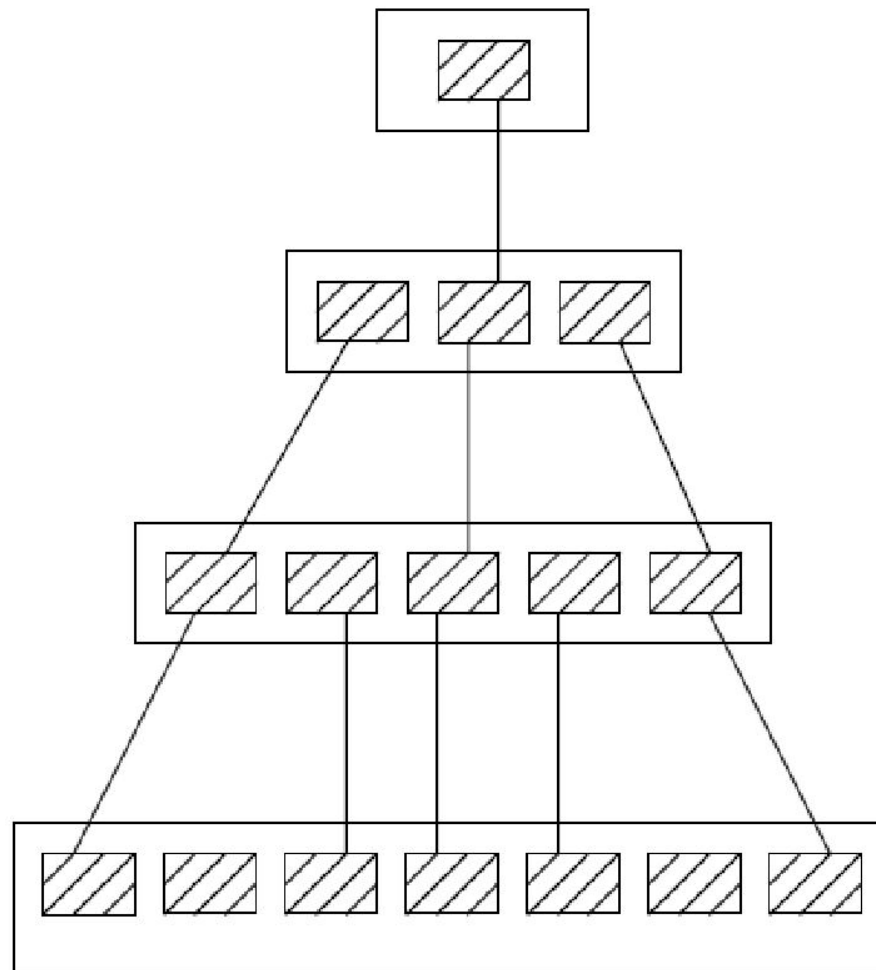


Abstraction

- The principle of abstraction implies that a problem can be simplified by omitting irrelevant details.
- In other words, the main purpose of abstraction is to consider only those aspects of the problem that are relevant for certain purpose and suppress other aspects that are not relevant for the given purpose.
- Once the simpler problem is solved, then the omitted details can be taken into consideration to solve the next lower level abstraction, and so on.
- Abstraction is a powerful way of reducing the complexity of the problem.



Abstraction



3rd abstraction

2nd abstraction

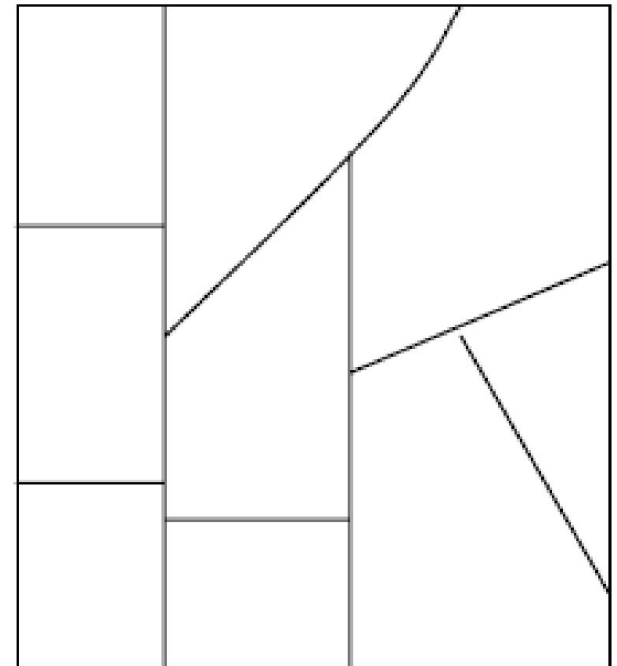
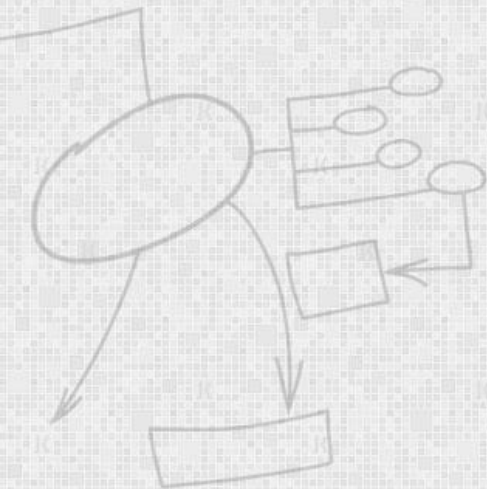
1st abstraction

Full Problem



Decomposition

- In this technique, a complex problem is divided into several smaller problems and then the smaller problems are solved one by one.



Decomposition

- However, in this technique any random decomposition of a problem into smaller parts will not help.
- The problem has to be decomposed such that each component of the decomposed problem can be solved independently and then the solution of the different components can be combined to get the full solution.
- A good decomposition of a problem should minimize interactions among various components.
- If the different subcomponents are interrelated, then the different components cannot be solved separately and the desired reduction in complexity will not be realized.

